ORBIT: Your Personal AI Action Agent

Product Overview, Architecture & Getting Started Guide

Target Track: Amazon Developer Hackathon — Alexa+ Track | Protocol: MCP over Streamable HTTP

1. What the Actual Project Does

ORBIT is an autonomous, context-aware personal AI action agent that transforms compound, multi-intent natural language requests into structured, validated, and verifiable real-world workflows.

Traditional conversational assistants often provide great suggestions, but they leave the execution to the user across disconnected applications (calendar apps, to-do lists, and grocery portals). Worse, typical agentic wrappers suffer from "hallucinated success"—claiming an action took place when an API timed out or failed silently.

The ORBIT Truthful Execution Guarantee

A tool execution is never marked successful simply because an LLM claims success. All state changes are committed through strict Model Context Protocol (MCP) tools and verified via authoritative read-backs directly from SQLite storage within 20 milliseconds.

The 8-Stage Core Operational Loop

Every user request passes through a deterministic, observable lifecycle:

- 1. UNDERSTAND: Decomposes natural language into intent, temporal bounds, and constraints.
- 2. RETRIEVE: Queries calendar, pending tasks, inventory, and preferences via MCP read tools.
- 3. PLAN: Resolves dependencies and schedules conflict-free operations.
- 4. VALIDATE: Performs server-side conflict detection and argument validation.
- 5. APPROVE: Gates all write actions behind cryptographic SHA-256 approval tokens.
- 6. EXECUTE: Invokes approved tools over real Streamable HTTP JSON-RPC 2.0.
- 7. VERIFY: Asserts database state directly with read-back queries before reporting status.
- 8. REMEMBER: Persists explicit user habits and confirmed preferences across sessions.

The 8 MCP Tools in the Catalogue

Domain capabilities are decoupled into an official MCP server with strict runtime Zod schemas:

[Read] get_schedule: Returns calendar blocks for date range, source, and timezone.
[Read] list_tasks: Fetches active/completed tasks filtered by status and priority.
[Write] create_task: Creates task with title, priority, due date, and mandatory idempotency key.
[Write] add_schedule_block: Allocates calendar block after validating against existing conflicts.
[Read] get_shopping_list: Retrieves current active grocery/shopping list items and quantities.
[Write] add_shopping_items: Appends validated grocery items while deduplicating active inventory.
[Read] get_preferences: Retrieves explicit, user-confirmed constraints, habits, and preferences.
[Write] save_preference: Persists user-confirmed preferences; explicitly blocks speculative inferences.

2. Architecture & Security Governance

ORBIT requires zero physical hardware. The entire solution operates locally or in the cloud using modern web standards, satisfying the Alexa+ software track:

System Topology

• Web Console (:5173): React + Vite dashboard displaying chat, plan previews, and live SSE logs.
• API Orchestrator (:3001): Node.js engine running the planner, gatekeeper, and verifier.
• MCP Server (:3002): Self-hosted Streamable HTTP JSON-RPC 2.0 endpoint with Server-Sent Events.
• SQLite State Engine: Embedded, synchronous, zero-dependency persistence via Node.js native DatabaseSync.

Core Safety & Governance Controls

• **Cryptographic Action Hash Binding:** Every proposed write computes sha256(canonicalPayload). Approval is bound to this exact hash. Any payload modification immediately invalidates approval (ERR_PAYLOAD_TAMPERED).

• **Approval Gatekeeper:** Any write tool execution attempted without prior human approval is blocked with HTTP 403 / ERR_APPROVAL_REQUIRED. Zero database records are created.

• **Server-Side Conflict Avoidance:** Calendar blocks that overlap with existing commitments (e.g. classes or meetings) are caught by collision algorithms (ERR_CONFLICT).

• **Credential Hygiene:** API keys and credentials live exclusively in server .env; zero secrets are ever emitted to client bundles, DOM, or logs.

3. How to Start and Run the Project

Set up, seed, test, and launch ORBIT using the following terminal commands:

Quickstart Commands

```
npm run setup     # Step 1: Install all workspace dependencies
npm run db:reset  # Step 2: Seed SQLite with clean demo scenario data
npm run test      # Step 3: Run 10-point automated acceptance suite
npm run dev       # Step 4: Start MCP (:3002), API (:3001), Web (:5173)
```

Accessing the Web Console Simulator

Open http://localhost:5173 in your browser. The top status bar will indicate active connections to the Streamable HTTP MCP Server and persistent SQLite state engine. Zero physical hardware required.

4. Interactive Demo Scenarios & Rehearsal

The Web Console includes 5 pre-configured one-click prompt chips at the bottom of the chat panel:

Scenario 1: Flagship 3-Intent Daily Planning [PRIMARY WORKFLOW]
Prompt: "Plan tomorrow using my saved preferences, reserve two hours for my ML project, and add missing groceries to my list."
Behavior: ORBIT inspects tomorrow's schedule (detects morning classes 9-12 and dinner at 18:30). It consults preferences and schedules the 2-hour ML block at 13:30 (conflict-free), reserves a 17:00 grocery errand, appends missing groceries, and creates a project task. Click "Approve All" to execute and view the green Read-Back Verification Report.

Scenario 2: Schedule Conflict Collision Detection [COLLISION AVOIDANCE]
Prompt: "Try to schedule a block that conflicts with an existing event and explain what you do."
Behavior: Attempts to schedule an event during the 9:00 AM class. ORBIT's collision algorithm detects the overlap and flags a warning notice (ERR_CONFLICT), demonstrating safe collision avoidance.

Scenario 3: Human-in-the-Loop Rejection Test [ZERO-WRITE SAFETY]
Prompt: "Reject a proposed task and prove that it was not created."
Behavior: ORBIT stages a proposal. Clicking the "Reject Step" button stops the action. The database is inspected to verify that zero records were created in SQLite.

Scenario 4: Add Groceries with Deduplication [INVENTORY SYNC]
Prompt: "Add these three groceries, but show me the exact changes before saving."
Behavior: Appends validated groceries with item counts and units, automatically deduplicating against active items.

Scenario 5: User Preference & Habit Memory [CROSS-SESSION PERSISTENCE]
Prompt: "Remember that I prefer focused study blocks of 50 minutes, and show me what you saved."
Behavior: Persists an explicit confirmed preference to the preferences table, viewable under the Memory tab in the Inspector.

5. Port & Service Reference Table

Port 5173 — ORBIT Web Console: Vite / React frontend with real-time SSE listener and dark theme.
Port 3001 — API Orchestrator: Express REST backend managing plans, gatekeeping, and read-back assertions.
Port 3002 — MCP Server: Streamable HTTP JSON-RPC 2.0 tool server (/mcp/message, /mcp/events).
data/orbit.sqlite — SQLite State Engine: Synchronous persistence for tasks, schedule blocks, groceries, preferences.